

# Relazione - Versione 1: Modello statistico a caratteri

## Introduzione

La Versione 1 è il primo modello funzionante del progetto Building LLM.

L'obiettivo è costruire da zero un modello linguistico semplice e interpretabile, usando esclusivamente Python standard e senza reti neurali o framework esterni.

Questa versione si concentra sul flusso fondamentale di un modello a caratteri:

```
Testo di training
↓
Copie di caratteri
↓
Conteggi delle transizioni
↓
Probabilità
↓
Campionamento
↓
Testo generato
```

La struttura è volutamente essenziale: ogni passaggio resta visibile e può essere controllato direttamente.

---

## Obiettivo della Versione 1

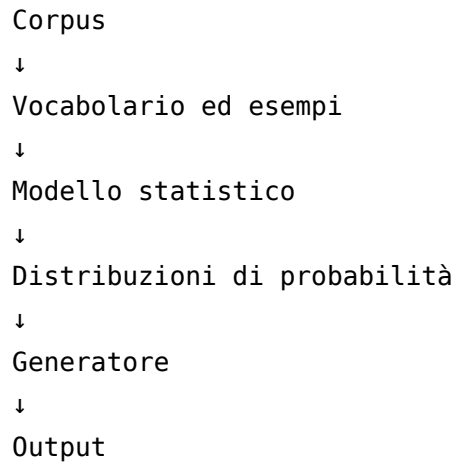
Il modello deve essere in grado di:

- leggere un piccolo corpus inglese;
- costruire il vocabolario dei caratteri;
- creare esempi formati da carattere corrente e carattere successivo;
- apprendere i conteggi delle transizioni;
- convertire i conteggi in probabilità;
- generare testo campionando un carattere alla volta.

Questa versione è la base didattica del progetto.

# Architettura

Il flusso completo della Versione 1 è:



Il modello usa come contesto un solo carattere. Per ogni carattere osservato, memorizza quali caratteri sono comparsi subito dopo e quante volte.

La generazione è autoregressiva: il carattere appena generato diventa il nuovo contesto per il passo successivo.

---

## Corpus e vocabolario

Il corpus è un breve testo inglese incorporato direttamente nel notebook. Non sono necessari file esterni.

Il vocabolario è l'insieme di tutti i simboli distinti presenti nel corpus. In un modello a caratteri, lettere, spazi, punteggiatura e ritorni a capo sono token.

```
vocabulary = sorted(set(corpus))
```

```
Vocabulary size: 27
```

```
'\n ,.abcdefghijklmnopqrstuvwxyz'
```

## Esempi di training

Ogni coppia di caratteri adiacenti costituisce un esempio. Il primo carattere è il contesto e il secondo è il target da prevedere.

Per il frammento model gli esempi sono:

```
m -> o
o -> d
d -> e
e -> l
```

Nel notebook gli esempi vengono costruiti accoppiando il corpus con una copia spostata di una posizione.

```
examples = list(zip(corpus, corpus[1:]))
```

Con 440 caratteri si ottengono 439 esempi, perché l'ultimo carattere non ha un carattere successivo.

---

## Modello statistico

L'addestramento consiste nel contare le transizioni osservate. Per ogni carattere corrente viene mantenuto un contatore dei possibili caratteri successivi.

```
for current_character, next_character in zip(text, text[1:]):
    counts[current_character][next_character] += 1
```

La tabella dei conteggi è il modello appreso. Non esistono ancora pesi neurali, embedding o ottimizzazione tramite gradienti.

## Probabilità

Per ottenere una distribuzione, ogni conteggio viene diviso per il totale delle transizioni osservate dallo stesso carattere corrente.

$$P(\text{next} \mid \text{current}) = \frac{\text{count}(\text{current} \rightarrow \text{next})}{\text{total counts}(\text{current})}$$

La somma delle probabilità relative allo stesso contesto è uguale a 1. Se un carattere non possiede transizioni, la funzione restituisce un dizionario vuoto.

Per il carattere m il corpus produce:

o: 4/18    a: 4/18    e: 4/18  
spazio: 2/18    p: 2/18    s: 2/18

---

## Campionamento

Il generatore non sceglie sempre lo stesso carattere. Campiona dalla distribuzione appresa: i caratteri con probabilità maggiore hanno più possibilità di essere selezionati.

```
random_generator.choices(  
    characters,  
    weights=weights,  
    k=1  
)[0]
```

Un seed locale rende il risultato riproducibile senza modificare lo stato casuale globale di Python.

## Generazione del testo

A partire da un carattere iniziale, il generatore ripete quattro operazioni:

- legge il carattere corrente;
- campiona il carattere successivo;
- aggiunge il carattere all'output;
- usa il nuovo carattere come contesto successivo.

La funzione controlla che la lunghezza richiesta sia almeno 1 e che il carattere iniziale sia stato osservato durante il training.

```
generate_text(model, start_character="i", length=300, seed=42)
```

Il testo prodotto può assomigliare localmente all'inglese, ma non mantiene una coerenza a lungo termine perché il contesto contiene un solo carattere.

---

## Test inclusi

Le asserzioni del notebook verificano:

- il numero corretto di esempi;
- la somma delle probabilità uguale a 1;
- la riproducibilità con lo stesso seed;
- la lunghezza richiesta dell'output.

L'esecuzione completa termina con il messaggio `All checks passed.`

# Cosa insegna la Versione 1

L'idea principale è:

osservare -> contare -> normalizzare -> campionare -> ripetere

La Versione 1 mostra che un modello linguistico può essere descritto come una distribuzione del simbolo successivo condizionata dal contesto disponibile.

Rende inoltre concreto il ciclo autoregressivo che verrà mantenuto, con contesti e modelli più potenti, nelle versioni successive.

---

## Limitazioni

Questa prima versione presenta limitazioni intenzionali:

- il contesto contiene un solo carattere;
- il corpus è piccolo e incorporato nel notebook;
- le transizioni mai osservate hanno probabilità zero;
- il modello non comprende parole, frasi o significato;
- la qualità del testo dipende interamente dai conteggi del corpus.

Queste limitazioni definiscono il punto di partenza del percorso.

---

## Conclusione

La Versione 1 è il primo modello funzionante di Building LLM. Trasforma un corpus in esempi, apprende una tabella di transizioni, costruisce distribuzioni di probabilità e genera nuovo testo un carattere alla volta.

Il risultato è semplice, completamente ispezionabile e coerente con l'obiettivo di costruire un modello linguistico dalle basi.